PEMROGRAMAN SISI SERVER

Pengenalan Framework Django

Dasar Django dan Docker



Ajib Susanto
Universitas Dian Nuswantoro

Tujuan Pembelajaran

1. Memahami konsep dan keunggulan framework Django
2. Menjelaskan arsitektur MTV dan alur request–response
3. Membuat project, app, view, dan template pertama
4. Menggunakan Model, migrasi, dan Django Admin
5. Membootstrap project Django dengan Docker dan Docker Compose

Apa itu Django?

Django adalah framework web open source berbasis Python untuk membangun aplikasi web dengan cepat dan rapi.

- Berjalan di sisi server: menerima request, memproses, lalu mengirim response
- Prinsip “batteries included”: autentikasi, ORM, admin, dan form sudah tersedia
- Dikembangkan komunitas dan dinaungi Django Software Foundation

2005 Rilis open source

Python Bahasa pemrograman

BSD Lisensi open source

Mengapa Django?

Cepat Dibangun

Fitur umum tersedia, fokus pada logika aplikasi

Aman

Proteksi CSRF, XSS, dan SQL injection bawaan

Skalabel

Dipakai situs ber-traffic tinggi, mudah dikembangkan

ORM Bawaan

Akses database dengan Python, tanpa menulis SQL

Admin Otomatis

Antarmuka kelola data dibuat dari model

Ekosistem Luas

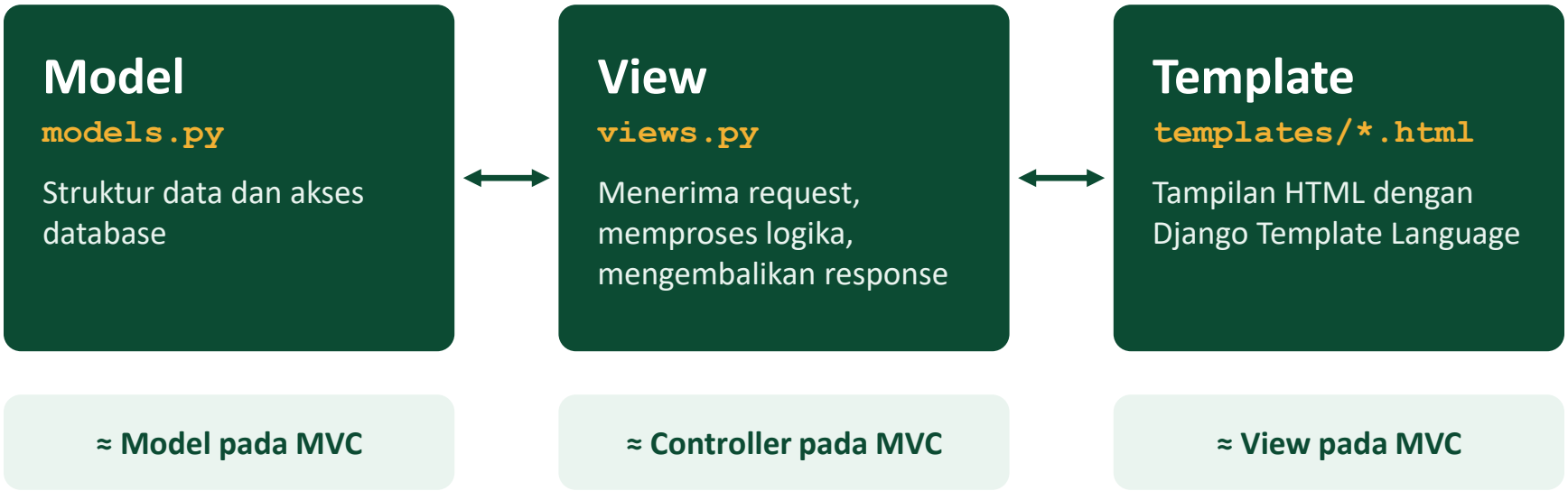
Banyak paket pihak ketiga dan dokumentasi lengkap

Django vs Framework Lain

Aspek	Django	Laravel	CodeIgniter 4
Bahasa	**Python**	PHP	PHP
Arsitektur	**MTV**	MVC	MVC
Akses data	**Django ORM**	Eloquent ORM	Model & Query Builder
Perintah CLI	**manage.py**	artisan	spark
Panel admin	**Bawaan**	Paket tambahan	Paket tambahan

Konsepnya serupa di semua framework. Yang berbeda: bahasa, konvensi, dan fitur bawaan.

Arsitektur MTV



Django berperan sebagai "controller": URL dispatcher meneruskan request ke view yang sesuai.

Alur Request–Response



Pengguna meminta halaman (HTTP request)

URL dicocokkan dengan pola path()

View memproses logika dan mengambil data

Data direnderr menjadi HTML

HTML dikirim kembali ke browser

Contoh alur

```
GET /produk/  →  urls.py  →  views.daftar  →  daftar.html  →  HTML
```

Instalasi Django

```
# 1. Buat virtual environment
python -m venv venv

# 2. Aktifkan (Windows)
venv\Scripts\activate
# Linux / macOS
source venv/bin/activate

# 3. Pasang Django
pip install django

# 4. Cek versi
python -m django --version
```

Isolasi
Paket tiap proyek terpisah, tidak saling bentrok.

Reproduksi
Catat dependensi di requirements.txt.

Versi
Samakan versi Python dengan yang didukung Django.

Project dan App

```
# Buat project
django-admin startproject tokokita
cd tokokita

# Buat app
python manage.py startapp produk

# Jalankan server
python manage.py runserver
```

Server berjalan di **`http://127.0.0.1:8000/`**

Project
Seluruh situs: konfigurasi dan URL utama

App
Modul fungsional (produk, akun, blog) yang bisa dipakai ulang

```
INSTALLED_APPS = [
    "produk",  # daftarkan app
]
```

Struktur Proyek

```
tokokita/
├── manage.py
├── tokokita/
│   ├── settings.py
│   ├── urls.py
│   ├── asgi.py
│   └── wsgi.py
└── produk/
    ├── models.py
    ├── views.py
    ├── admin.py
    ├── apps.py
    ├── tests.py
    └── migrations/
```

`manage.py`	Perintah CLI Django
`settings.py`	Konfigurasi project
`urls.py`	Routing URL
`models.py`	Definisi model data
`views.py`	Logika penanganan request

View dan URL Routing

produk/views.py

```
from django.http import HttpResponse

def halo(request):
    return HttpResponse("Halo, Django!")
```

produk/urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path("", views.halo, name="halo"),
]
```

tokokita/urls.py

```
from django.urls import path, include

urlpatterns = [
    path("produk/", include("produk.urls")),
]
```

Hasil di browser

http://127.0.0.1:8000/produk/

Halaman menampilkan: Halo, Django!

Template Django

produk/views.py

```python
from django.shortcuts import render

def daftar(request):
    ctx = {
        "judul": "Daftar Produk",
        "items": ["Kopi", "Teh"],
    }
    return render(
        request, "produk/daftar.html", ctx
    )
```

templates/produk/daftar.html

```
<h1>{{ judul }}</h1>
<ul>
  {% for item in items %}
    <li>{{ item }}</li>
  {% endfor %}
</ul>
```

`{{ variabel }}`
mencetak nilai

`{% tag %}`
logika: for, if, dll.

`{# komentar #}`
tidak tampil di HTML

Model dan ORM

```python
from django.db import models

class Produk(models.Model):
    nama = models.CharField(
        max_length=100)
    harga = models.PositiveIntegerField()
    stok = models.IntegerField(default=0)
    dibuat = models.DateTimeField(
        auto_now_add=True)

    def __str__(self):
        return self.nama
```

Class Produk
→ tabel produk_produk

Atribut (field)
→ kolom tabel

Objek (instance)
→ baris data

Migrasi Database

1. **Ubah model**
   Tambah atau ubah class di models.py

2. **`makemigrations`**
   Membuat file migrasi dari perubahan model

3. **`migrate`**
   Menerapkan migrasi ke database

```
python manage.py makemigrations
python manage.py migrate
```

Pengaturan database

- Bawaan: SQLite (berkas db.sqlite3)
- Bisa diganti ke MySQL atau PostgreSQL lewat DATABASES di settings.py
- Setiap perubahan model memerlukan migrasi baru

Query dengan ORM

Operasi	Metode ORM
Create	`create()`
Read	`all()` `filter()` `get()`
Update	`save()`
Delete	`delete()`

Hasil query berupa QuerySet yang bisa difilter berantai.

```
python manage.py shell

>>> from produk.models import Produk
>>> Produk.objects.create(
...     nama="Kopi", harga=15000)
>>> Produk.objects.all()
>>> Produk.objects.filter(
...     harga__lt=20000)
>>> p = Produk.objects.get(id=1)
>>> p.harga = 17000
>>> p.save()
>>> p.delete()
```

Django Admin

Buat akun admin

```
python manage.py createsuperuser
```

produk/admin.py

```
from django.contrib import admin
from .models import Produk

admin.site.register(Produk)
```

Catatan: jalankan migrate dulu agar tabel pengguna tersedia.



Login di /admin/
Masuk dengan akun superuser



CRUD otomatis
Tambah, ubah, hapus data tanpa menulis view



Autentikasi & izin
Kelola pengguna, grup, dan hak akses

Perintah manage.py

`runserver`
Menjalankan server pengembangan

`startapp`
Membuat app baru dalam project

`makemigrations`
Membuat file migrasi dari model

`migrate`
Menerapkan migrasi ke database

`createsuperuser`
Membuat akun administrator

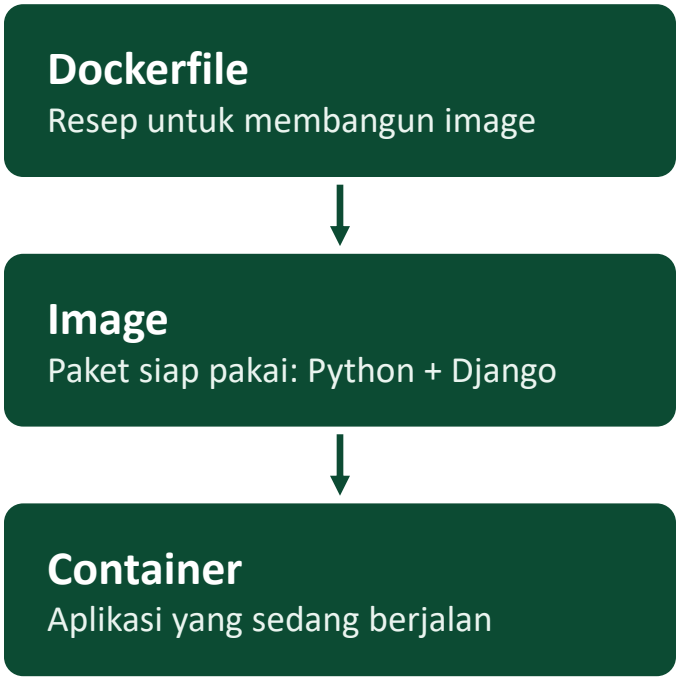
`shell`
Konsol Python dengan konteks Django

Semua perintah diawali `python manage.py <perintah>`

Docker untuk Django

Docker membungkus aplikasi Django beserta Python, dependensi, dan konfigurasinya dalam container.

- Lingkungan sama di laptop dosen, mahasiswa, dan server
- Tidak perlu memasang Python dan PostgreSQL di komputer
- Layanan lain (database, cache) mudah ditambah lewat Docker Compose



Bootstrap Project via Docker

```
# 1. Siapkan berkas awal
mkdir tokokita && cd tokokita
# requirements.txt, Dockerfile,
# compose.yaml, .env

# 2. Bangun image
docker compose build

# 3. Buat project di dalam container
docker compose run --rm web \
  django-admin startproject config .

# 4. Buat app
docker compose run --rm web \
  python manage.py startapp produk
```

`run --rm`
Jalankan satu perintah di container baru, lalu hapus

`titik (.)`
Project dibuat di /app, terhubung ke folder host

`volume .:/app`
File hasil langsung muncul di komputer Anda

Dockerfile untuk Django

```
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000
CMD ["python", "manage.py", "runserver", "0.0.0.0:8000"]
```

FROM
Image dasar ringan dengan Python 3.12

COPY
Salin requirements.txt dulu (cache layer)

RUN pip install
Pasang dependensi saat build

0.0.0.0:8000
Server bisa diakses dari luar container

Docker Compose: web + db

```yaml
services:
  web:
    build: .
    ports:
      - "8000:8000"
    volumes:
      - .:/app
    env_file: .env
    depends_on:
      - db
  db:
    image: postgres:16
    env_file: .env
    volumes:
      - pgdata:/var/lib/postgresql/data
volumes:
  pgdata:
```

web
Aplikasi Django, dibangun dari Dockerfile

db
PostgreSQL dari image resmi

ports
8000:8000 → akses lewat localhost:8000

volumes
Bind mount kode dan volume pgdata untuk data DB

Konfigurasi lewat .env

config/settings.py

```python
import os

SECRET_KEY = os.environ["DJANGO_SECRET_KEY"]
DEBUG = os.environ.get("DJANGO_DEBUG") == "1"
ALLOWED_HOSTS = ["localhost", "127.0.0.1"]

DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.postgresql",
        "NAME": os.environ["POSTGRES_DB"],
        "USER": os.environ["POSTGRES_USER"],
        "PASSWORD": os.environ["POSTGRES_PASSWORD"],
        "HOST": os.environ["POSTGRES_HOST"],
        "PORT": os.environ["POSTGRES_PORT"],
    }
}
```

.env

```
POSTGRES_DB=tokokita
POSTGRES_USER=tokokita
POSTGRES_PASSWORD=rahasia
POSTGRES_HOST=db
POSTGRES_PORT=5432
DJANGO_SECRET_KEY=ganti-saja
DJANGO_DEBUG=1
```

Penting: POSTGRES_HOST berisi nama service (db), bukan localhost.

Menjalankan di Container

```
# Jalankan semua service
docker compose up --build

# Di terminal lain
docker compose exec web \
  python manage.py migrate
docker compose exec web \
  python manage.py createsuperuser

# Log dan berhenti
docker compose logs -f web
docker compose down
```

localhost:8000
Halaman awal Django tampil

localhost:8000/admin/
Masuk dengan akun superuser

down -v
Menghapus volume: data database ikut hilang

Contoh Kasus: Katalog Produk

```
tokokita/
├── compose.yaml
├── Dockerfile
├── .env
├── .dockerignore
├── requirements.txt
├── manage.py
├── config/
│   └── settings.py
└── produk/
```

1. Bootstrap: build image, startproject, startapp
2. Atur settings.py: database dari environment
3. Jalankan: docker compose up --build
4. migrate dan createsuperuser lewat exec
5. Buat model Produk, migrasi, daftarkan di Admin

Tips dan Kesalahan Umum

Host database

Gunakan db sebagai HOST, bukan localhost

Tunggu DB siap

depends_on tidak menunggu DB siap; pakai healthcheck

Data di volume

Volume pgdata menyimpan data; down -v menghapusnya

.dockerignore

Abaikan venv, .git, dan __pycache__

Bukan produksi

runserver hanya untuk pengembangan; gunakan gunicorn

Izin file Linux

File dari container bisa milik root; jalankan dengan --user

Latihan Praktikum

1. Buat project “perpus” dan app “buku”
2. Buat model Buku (judul, penulis, tahun_terbit), lalu migrasi
3. Daftarkan ke Admin dan isi 3 data buku
4. Buat view dan template untuk menampilkan daftar buku
5. Jalankan project “perpus” dengan Docker Compose (Django + PostgreSQL)

Referensi

docs.djangoproject.com

Tutorial resmi: Writing your first Django app

Ringkasan



Django adalah framework Python yang lengkap untuk membangun aplikasi web



Arsitektur MTV, dengan URL sebagai pintu masuk setiap request



Satu project berisi banyak app, dikelola lewat manage.py



Model + migrasi + Admin = kelola data dengan cepat



Docker Compose menjalankan Django + PostgreSQL tanpa instal lokal



Konfigurasi lewat .env, host database = nama service

Terima kasih. Ada pertanyaan?